

Cahier des charges

Projet de gestion des flottes

Wilaya de la Région Souss-Massa
Préfecture d'Agadir Ida Outanane

Organisation publique

July 27, 2026

Table des matières

1 Utilisateurs, rôles et circuit de validation

La gestion de flotte s'appuie sur un circuit hiérarchique interne à l'organisation (la Wilaya), complété par une interaction externe avec l'opérateur télécom via une interface applicative dédiée. La structure organisationnelle repose sur des directions/services, chacun pouvant comporter plusieurs niveaux de supervision. La configuration précise de cette structure (nombre de directions, rattachement des utilisateurs, désignation des superviseurs) n'est pas figée dans l'architecture : elle est configurable par l'administrateur système via la gestion des rôles et des directions.

1.1 Acteurs et rôles du système

Cette section détaille les différents acteurs intervenant dans le circuit de traitement des demandes, ainsi que la nature concrète des actions associées à chacun.

1.1.1 Utilisateur (agent)

C'est l'utilisateur final de l'application, rattaché à une direction donnée. Il est à l'origine de toute demande liée à une ligne téléphonique ou à un service télécom de son périmètre. Concrètement, il peut soumettre trois types de demandes :

- **Nouvelle ligne** : demande de création et d'attribution d'une nouvelle ligne téléphonique (ou d'un nouveau service), généralement destinée à un collaborateur de sa direction.
- **Modification** : demande de changement sur une ligne existante — par exemple un changement de forfait, une mise à jour des options souscrites, un changement de titulaire, ou toute autre altération des caractéristiques du service sans le supprimer.
- **Résiliation** : demande d'arrêt définitif d'une ligne ou d'un service (par exemple lors du départ d'un collaborateur ou de la fin d'un besoin).

Après réception physique d'une ligne (à la suite du traitement d'une demande de type nouvelle ligne), l'utilisateur est également invité à enregistrer/confirmer son numéro dans l'application, afin d'établir l'association entre son compte et la ligne qui lui a été attribuée.

1.1.2 Superviseur de direction

Il constitue le premier échelon de supervision au sein d'une direction. Il dispose des mêmes droits qu'un utilisateur standard (il peut lui-même soumettre des demandes pour son propre usage) et supervise, en complément, un ensemble d'utilisateurs rattachés à sa direction.

Le superviseur de direction est automatiquement informé de toute demande soumise par les utilisateurs qu'il supervise. Cette information est faite à titre consultatif : il **n'exerce pas de pouvoir de validation** sur ces demandes. Il ne constitue donc pas un filtre bloquant dans le circuit — son rôle se limite à la visibilité et au suivi de l'activité de son périmètre.

1.1.3 Responsable de direction

Il constitue le niveau hiérarchique supérieur au superviseur de direction au sein d'une même direction. Il dispose des mêmes droits qu'un utilisateur standard et qu'un superviseur de direction (soumission de demandes propres, visibilité sur l'inventaire et l'activité de la direction), et détient en complément le **pouvoir de validation interne** des demandes.

C'est à ce niveau que se joue la seule décision humaine interne du circuit : le responsable de direction peut soit **rejeter définitivement** une demande (fin du processus, l'auteur en est

notifié), soit la **valider**, ce qui déclenche automatiquement sa transmission vers l'opérateur télécom concerné.

Avant de valider, le responsable de direction dispose d'un espace de travail lui permettant d'ajuster le contenu de la demande — par exemple préciser ou corriger le type de ligne demandé, les options souscrites, ou tout autre détail pertinent — sans altérer la demande originale telle que soumise par son auteur. Ces ajustements peuvent être modifiés librement tant que la demande n'a pas été validée ; une fois la validation effectuée, le contenu ajusté est figé et c'est cette version, et non la demande d'origine, qui est transmise à l'opérateur télécom.

1.1.4 Opérateur télécom

Contrairement à un rôle interne à l'organisation, l'opérateur télécom est un acteur externe disposant de son propre accès applicatif, dédié et cloisonné. Cet accès lui permet de consulter les demandes qui lui ont été transmises (et uniquement celles-ci) et d'y répondre en indiquant un statut de traitement (en cours, terminé, refusé) accompagné, le cas échéant, d'un message.

Le traitement effectif de la demande (activation d'une ligne, application d'une modification, résiliation d'un contrat) se déroule dans le système propre de l'opérateur, en dehors de l'application : cette dernière ne sert que de canal de transmission et de réception du résultat. Lorsque l'opérateur indique qu'une demande est terminée, l'inventaire de l'organisation est mis à jour automatiquement en conséquence (création, modification ou désactivation de la ligne concernée).

1.1.5 Administrateur système

Ce rôle est transversal et ne participe pas directement au circuit métier des demandes (nouvelle ligne / modification / résiliation). Il est responsable de la configuration et de la gouvernance de l'application elle-même :

- **Gestion des rôles** : définir les différents profils utilisateurs existants dans le système (utilisateur, superviseur de direction, responsable de direction, administrateur).
- **Gestion des permissions** : déterminer quelles actions chaque rôle est autorisé à effectuer (soumettre, superviser, valider, configurer...).
- **Attribution des responsabilités** : associer un utilisateur réel à un rôle donné, et le rattacher à la bonne direction dans la structure organisationnelle.
- **Gestion des accès opérateur** : créer et révoquer les accès applicatifs dédiés aux opérateurs télécom partenaires.

En complément de ces responsabilités de configuration, l'administrateur dispose d'un contrôle total sur le système et peut, si nécessaire, intervenir directement sur l'inventaire ou le circuit des demandes.

1.1.6 Logique d'ensemble du circuit

Le circuit repose sur une validation hiérarchique automatisée à un seul point de décision interne : toute demande soumise par un utilisateur ou un superviseur de direction est portée à la connaissance du superviseur de direction et du responsable de direction de sa direction, mais seul ce dernier dispose du pouvoir de validation ou de rejet. Une fois validée, la demande est transmise à l'opérateur télécom concerné via son propre accès applicatif, sans intervention manuelle supplémentaire côté organisation. Le retour de l'opérateur (statut et message) est automatiquement répercuté sur l'inventaire lorsque la demande est marquée comme terminée, puis notifié à l'auteur de la demande ainsi qu'au superviseur et au responsable de sa direction.

L'administrateur système, lui, ne se situe pas dans ce flux de demandes : il agit en amont, sur la structure même du système (qui a le droit de faire quoi, et à quel niveau), ainsi qu'en dernier recours en cas d'intervention exceptionnelle.

1.2 Cycle de vie d'une demande

Une demande suit les transitions d'état suivantes :

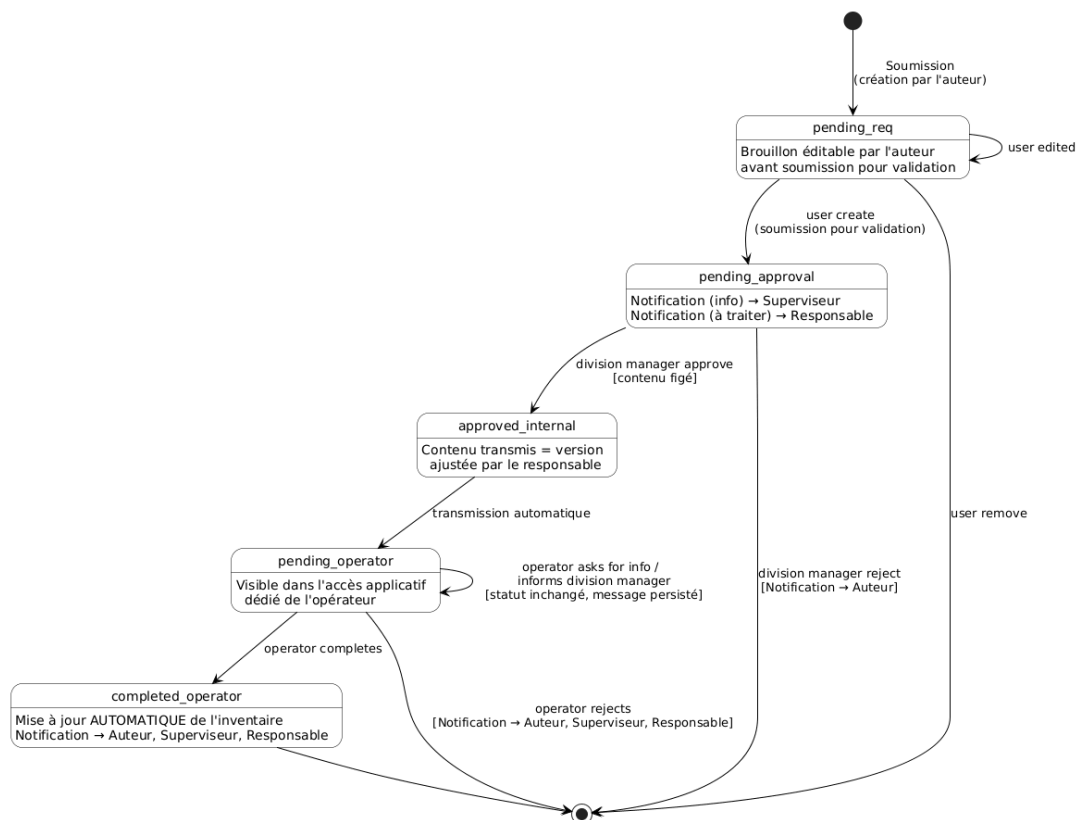


Figure 1: Cycle de vie d'une demande (diagramme d'états-transitions)

1. **En attente de validation** : créée par un utilisateur ou un superviseur de direction, la demande notifie le superviseur de direction (à titre informatif) et le responsable de direction (pour décision) de la direction concernée.
2. **Instruction** : le responsable de direction peut, à ce stade, ajuster librement le contenu de la demande (détails, options, précisions) sans que cela modifie la demande originale ni ne déclenche de notification ; cette phase de travail peut comporter plusieurs révisions successives avant la décision finale.
3. **Rejetée en interne** ou **Validée en interne**, selon la décision du responsable de direction. Un rejet met fin au processus et notifie l'auteur. Une validation fige le contenu ajusté et déclenche l'envoi de cette version à l'opérateur télécom concerné.
4. **Transmise à l'opérateur** : la demande, dans sa version validée, devient visible dans l'accès applicatif dédié de l'opérateur concerné.
5. **En cours de traitement** : l'opérateur peut engager un échange de messages avec le responsable de direction — seul interlocuteur interne habilité à ce stade — sans que ces échanges ne modifient le statut de la demande. Chaque message, dans un sens comme

dans l'autre, est conservé de façon persistante et reste consultable dans l'historique de la demande.

6. **Terminée** ou **refusée** par l'opérateur, avec un message optionnel joint à sa décision finale — cette décision peut intervenir directement après la transmission de la demande, ou à l'issue de la phase d'échanges.
7. Si la demande est marquée **terminée**, l'inventaire est mis à jour automatiquement à partir de la version validée par le responsable de direction (création, modification ou désactivation de la ligne selon le type de demande), et l'auteur ainsi que le superviseur et le responsable de sa direction sont notifiés du dénouement.

1.3 Notifications

Les acteurs du circuit sont tenus informés de l'avancement des demandes par un système de notifications, déclenché à chaque transition significative : soumission d'une demande (le superviseur de direction et le responsable de direction en sont notifiés), et réponse de l'opérateur télécom (l'auteur de la demande, le superviseur et le responsable de sa direction en sont notifiés). Une notification similaire est déclenchée lors d'un changement d'état d'une ligne déjà en service (par exemple, une ligne signalée en panne).

Ces notifications constituent un mécanisme d'alerte ponctuel : elles signalent qu'un événement a eu lieu, sans se substituer à la donnée elle-même.

1.4 Historique et justification des demandes

Chaque transition d'état d'une demande peut être accompagnée d'un message — une justification de rejet apportée par le responsable de direction, ou un commentaire de l'opérateur télécom expliquant pourquoi une demande reste en attente, est refusée, ou nécessite un complément d'information. Contrairement aux notifications, ces messages sont **conservés de façon persistante** et rattachés à la demande concernée : ils ne sont pas consommés une seule fois puis perdus, mais restent consultables à tout moment dans l'historique de la demande, indépendamment du fait que la notification associée ait déjà été vue ou non.

Cette persistance s'applique également au contenu de la demande elle-même : la version originale soumise par son auteur et la version éventuellement ajustée par le responsable de direction sont toutes deux conservées distinctement, ce qui permet de retracer précisément ce qui a été demandé, ce qui a été effectivement transmis à l'opérateur télécom, et par qui chaque ajustement a été apporté.

1.5 Communication externe (opérateur télécom)

La communication avec l'opérateur télécom se fait exclusivement via son accès applicatif dédié, cloisonné aux seules demandes qui lui sont destinées. Lorsqu'il répond à une demande, l'opérateur indique un statut de traitement (en cours, terminé, refusé) accompagné, le cas échéant, d'un message, qui est intégré à l'historique de la demande selon les mêmes principes que les échanges internes. Aucune communication externe (email, téléphone) n'est requise dans le cadre du circuit de traitement des demandes.

Ce cahier des charges précise les modalités fonctionnelles du système (acteurs, rôles, circuit de validation, cycle de vie des demandes, notifications). Le contexte, les objectifs et le périmètre du projet sont définis dans le document Expression de besoins associé.

2 Architecture technique

Cette section décrit l'architecture logicielle retenue pour la réalisation du système décrit dans les sections précédentes. Elle traduit en composants techniques le circuit fonctionnel (acteurs, rôles, cycle de vie des demandes) exposé plus haut.

2.1 Vue d'ensemble

Le système est organisé en microservices autour de trois domaines métier distincts, exposés via une passerelle API unique et communiquant de manière asynchrone via un bus d'événements pour les besoins de notification. Les données sont hébergées sur une instance PostgreSQL unique, partitionnée en schémas dédiés et cloisonnés par service.

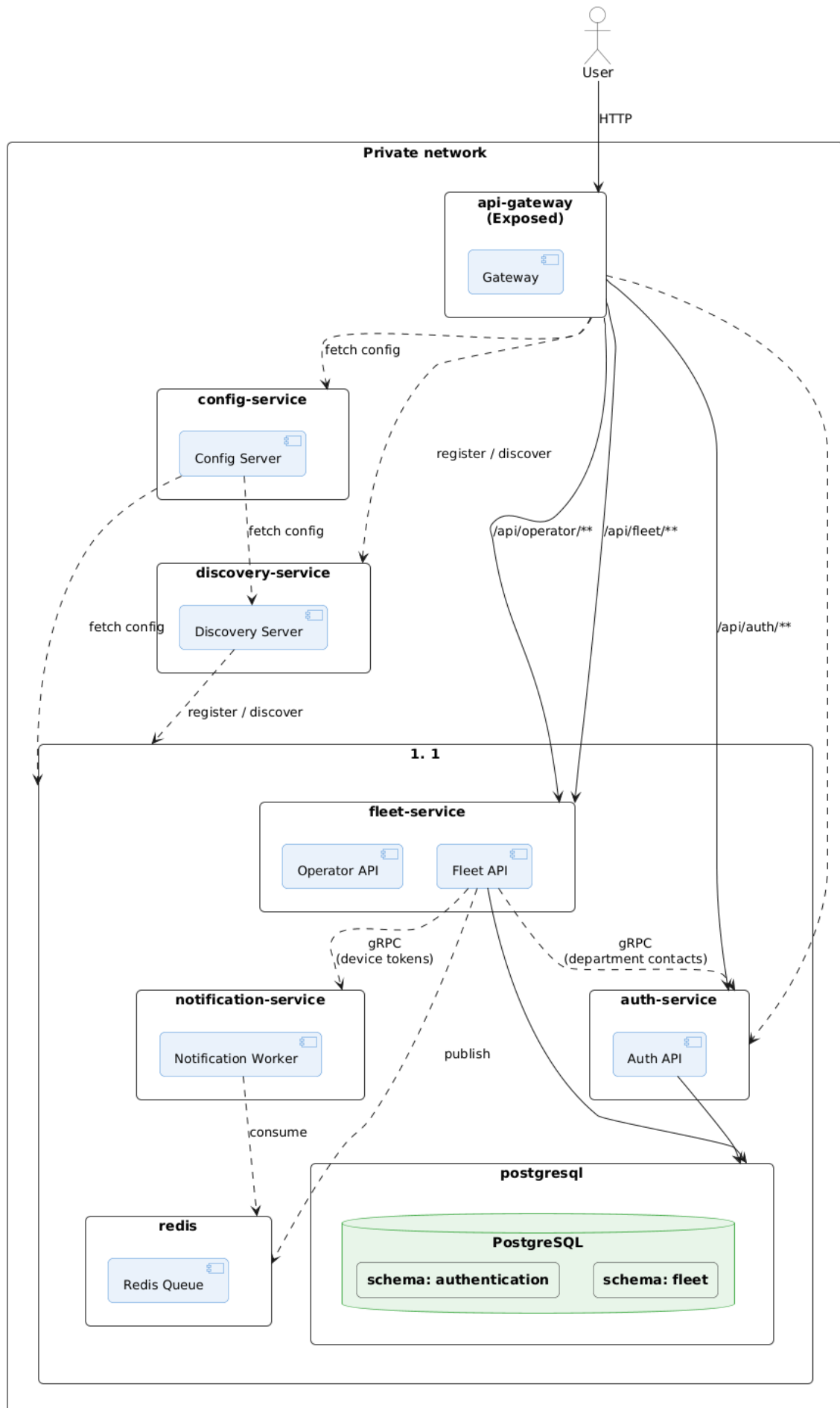


Figure 2: Architecture globale du système et flux inter-services (Docker Network)

2.2 Composants

2.2.1 API Gateway

Point d'entrée unique de l'application pour l'utilisateur. La passerelle route les requêtes vers les services appropriés selon le préfixe d'URL :

- `/api/auth/` vers **auth-service** (authentification : connexion, rafraîchissement de session) ;
- `/operator/` vers **fleet-service**, exclusivement pour les requêtes porteuses d'une revendication (*claim*) JWT `role=OPERATOR`, correspondant à l'accès applicatif dédié de l'opérateur télécom (cf. section 1.1.4) ;
- `/api/` vers les autres services applicatifs.

La passerelle récupère périodiquement le jeu de clés publiques (JWKS) auprès d'**auth-service** afin de valider localement la signature des jetons JWT présentés par les clients, sans appel systématique au service d'authentification pour chaque requête.

2.2.2 auth-service

Service responsable de l'authentification et de la gestion des identités. Il expose une *Auth API* et possède en propriété exclusive le schéma **authentication** de la base PostgreSQL, contenant les utilisateurs, les directions, les rôles (y compris le rôle **OPERATOR**, associé à un identifiant d'opérateur), ainsi que la hiérarchie de supervision décrite en section 1.1. Aucune jointure directe inter-schéma n'est autorisée : les autres services n'accèdent à ces données que via des appels applicatifs dédiés (gRPC), avec un rôle de base de données restreint.

Deux consommateurs internes sollicitent **auth-service** en gRPC :

- **fleet-service** y résout les superviseurs et responsables d'une direction donnée (à partir d'un `departmentId`), afin de déterminer les destinataires internes d'une demande ;
- **notification-service** y résout uniquement les jetons d'appareil (*device tokens*) nécessaires à l'envoi des notifications push, les identifiants `authorId` et `departmentId` étant déjà présents dans le contenu de l'événement reçu.

Modèle de données Le schéma **authentication** porte les entités **User**, **Department** et **Operator**, reliées selon le diagramme suivant.

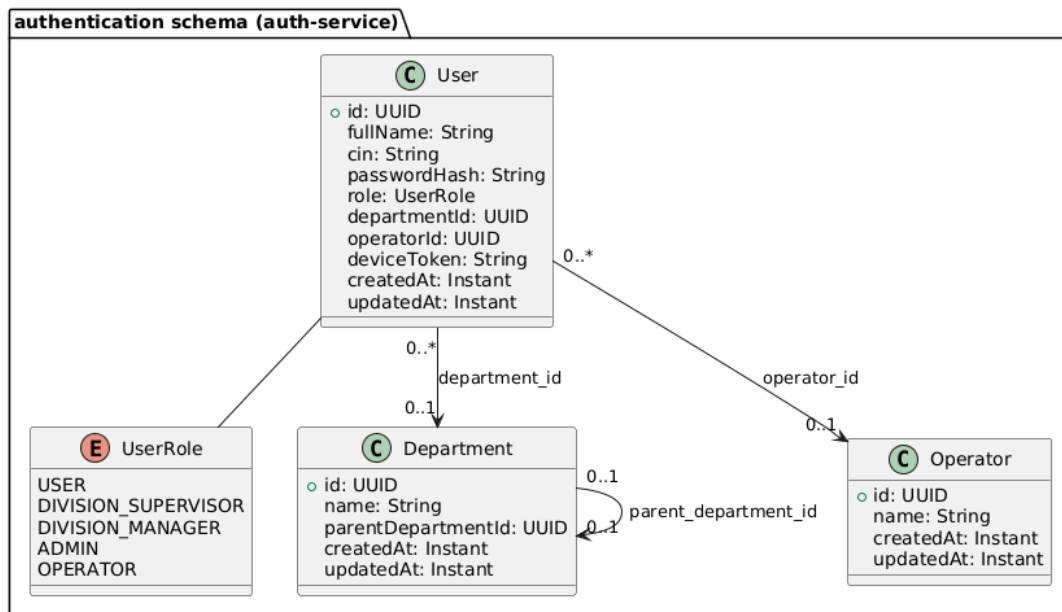


Figure 3: Schéma authentication (auth-service)

DTOs – Auth API (REST) L'Auth API expose les objets de transfert suivants pour les opérations de connexion, d'inscription et de gestion de session.

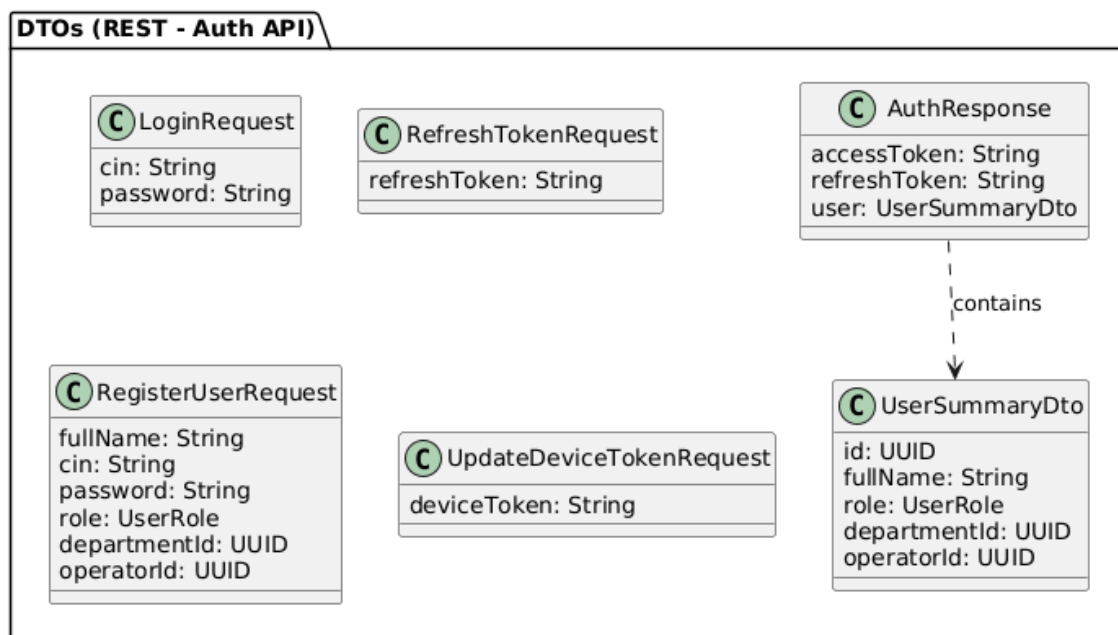


Figure 4: DTOs REST – Auth API

DTOs – gRPC internes Les échanges gRPC internes avec **fleet-service** et **notification-service** reposent sur les DTOs suivants.

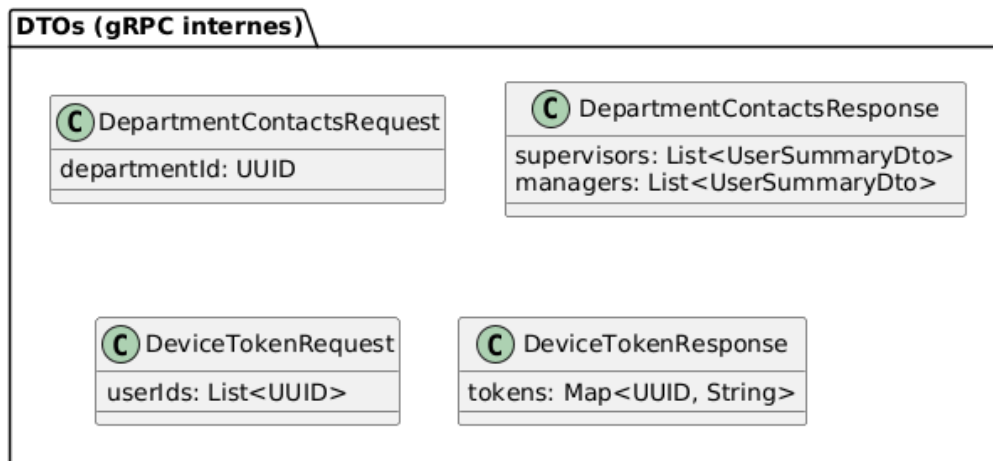


Figure 5: DTOs gRPC internes (auth-service)

2.2.3 fleet-service

Service cœur du métier, portant la logique de gestion des lignes, des demandes et de leur cycle de vie décrit en section 1.2. Il possède en propriété exclusive le schéma **fleet**, contenant les entités **Line**, **Request**, **RequestReview** et **RequestOperatorResponse**, gérées dans une même transaction locale afin de garantir la cohérence du cycle de vie d'une demande.

Il expose deux interfaces distinctes :

- la *Fleet API*, accessible via `/api/`, utilisée par les utilisateurs, superviseurs et responsables de direction pour soumettre, consulter, ajuster et valider les demandes ;
- l'*Operator API* (`/operator/**`), interface cloisonnée dédiée à l'opérateur télécom (section 1.1.4), filtrée par l'identifiant d'opérateur (`operatorId`) porté par le claim JWT, garantissant qu'un opérateur ne voit que les demandes qui lui sont destinées.

À chaque transition significative du cycle de vie d'une demande, **fleet-service** publie un événement dans **Redis** :

- `request.created` et `request.rejected_internal` lors de la soumission et du rejet interne d'une demande ;
- `request.resolved` lors de la réponse finale de l'opérateur, enrichi des identifiants `authorId` et `departmentId`.

Cet enrichissement systématique des événements publiés dispense **notification-service** de devoir interroger **auth-service** pour obtenir ces informations : seule la résolution des jetons d'appareil y reste nécessaire.

Modèle de données Le schéma **fleet** est entièrement découplé du schéma **authentication** : aucune clé étrangère cross-schéma n'existe. Les identifiants `authorId`, `departmentId`, `operatorId` et `assignedUserId` sont des références logiques, résolues à la demande via gRPC auprès de **auth-service**.

Pas de FK cross-schema.
authorid / departmentid / operatorid /
assignedUserId sont des references logiques,
resolues via gRPC vers identity-service.

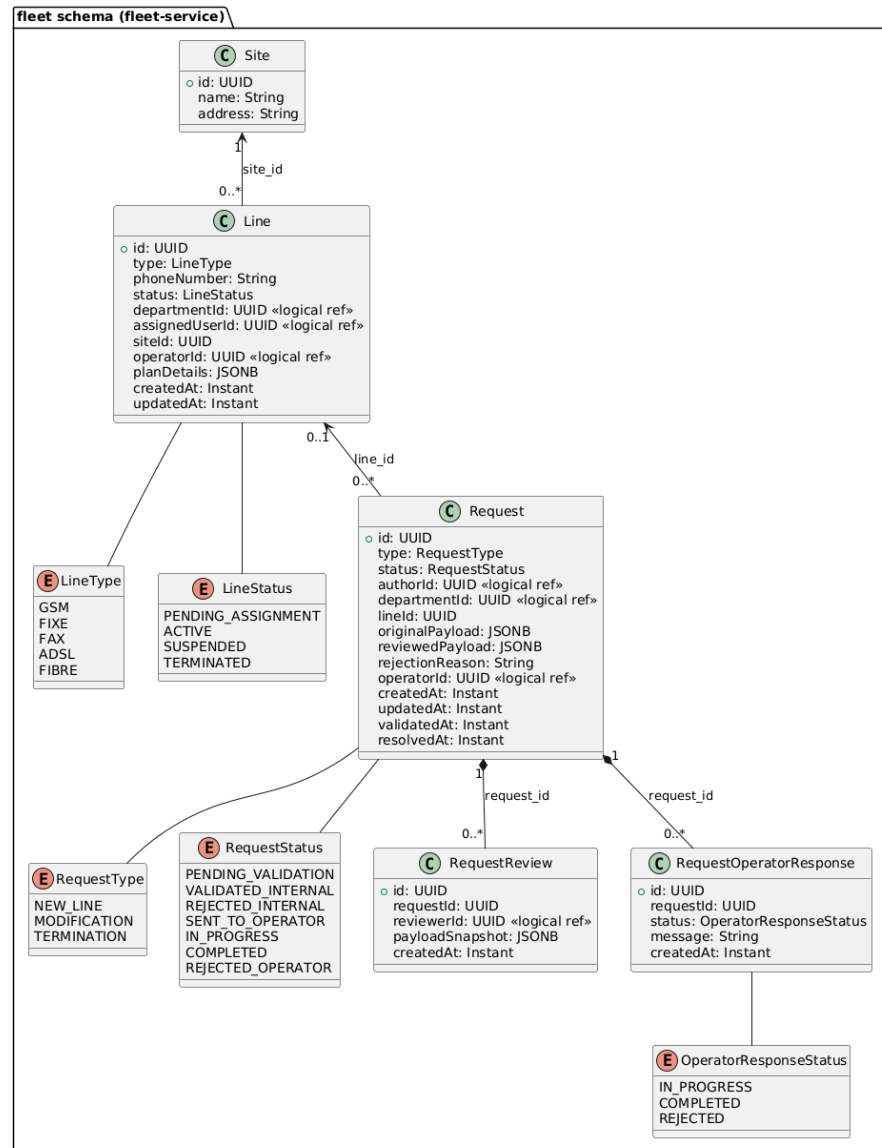


Figure 6: Schéma **fleet** (fleet-service)

DTOs – Fleet API (REST) L'interface applicative principale expose les DTOs suivants pour la création, la consultation, la révision et la validation des demandes, ainsi que pour la gestion des lignes.

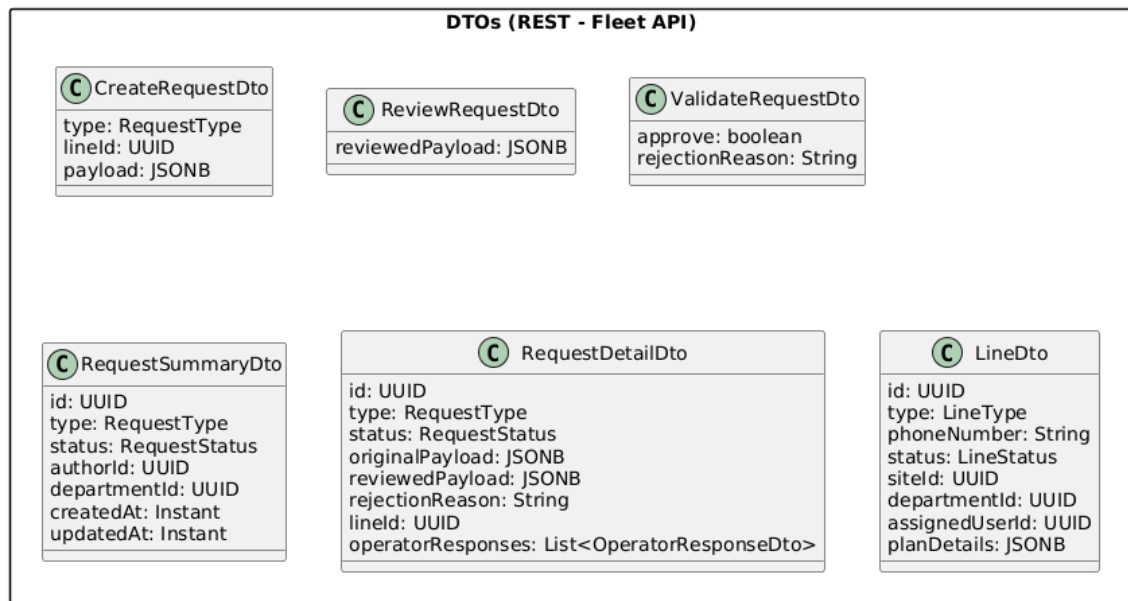


Figure 7: DTOs REST – Fleet API

DTOs – Operator API (REST) L'interface cloisonnée dédiée à l'opérateur télécom expose un sous-ensemble restreint des informations d'une demande, ainsi que les objets nécessaires à la réponse de l'opérateur.

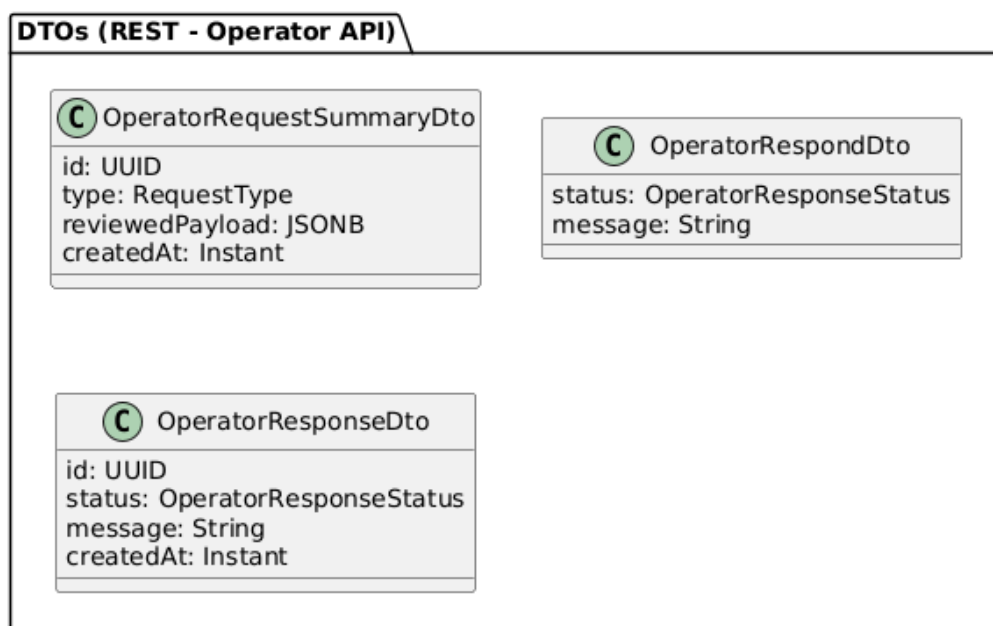


Figure 8: DTOs REST – Operator API

DTOs – gRPC vers auth-service Pour la résolution des superviseurs et responsables d'une direction, **fleet-service** consomme l'interface gRPC exposée par **auth-service**.

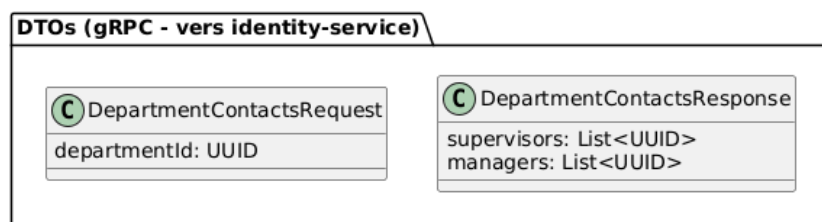


Figure 9: DTOs gRPC – vers auth-service

2.2.4 notification-service

Service asynchrone, sans exposition directe via la passerelle API, chargé de la production des notifications décrites en section 1.3. Le *Notification Worker* consomme en continu les événements publiés sur **Redis** par **fleet-service** et déclenche l'envoi des notifications aux destinataires concernés (auteur, superviseur, responsable de direction), après résolution des jetons d'appareil auprès d'**auth-service**.

2.2.5 Stack technique

La réalisation du système s'appuie sur l'écosystème Spring, complété par des composants dédiés à la communication inter-services et à la diffusion des notifications.

Framework applicatif Chaque microservice (**api-gateway**, **auth-service**, **fleet-service**, **notification-service**) est développé en **Java** sur la base de **Spring Boot**, garantissant une configuration homogène du cycle de vie applicatif, de l'injection de dépendances et de l'exposition des API REST via **Spring Web**.

Passerelle et routage L'**API Gateway** est implémentée avec **Spring Cloud Gateway**, qui assure le routage des requêtes selon le préfixe d'URL (`/api/auth/`, `/operator/`, `/api/`) ainsi que la validation locale des jetons JWT à partir du jeu de clés publiques (JWKS) récupéré périodiquement auprès d'**auth-service**.

Sécurité L'authentification et le contrôle d'accès reposent sur **Spring Security**, configuré pour la validation des jetons **JWT** (émission et rafraîchissement côté **auth-service**, vérification de signature côté passerelle et services applicatifs). Les autorisations sont dérivées des rôles (`USER`, `DIVISION_SUPERVISOR`, `DIVISION_MANAGER`, `ADMIN`, `OPERATOR`) portés par les claims du jeton, garantissant notamment le cloisonnement de l'*Operator API* par `operatorId`.

Découverte de services et configuration **discovery-service** est implémenté avec **Netflix Eureka**, permettant l'enregistrement et la découverte dynamique des instances de chaque microservice. La configuration centralisée est assurée par **config-service** (**Spring Cloud Config**), interrogé au démarrage par l'ensemble des services applicatifs pour récupérer leurs paramètres d'environnement.

Communication inter-services Les échanges synchrones internes (résolution des superviseurs et responsables de direction, résolution des jetons d'appareil) sont réalisés en **gRPC**, choisi pour sa performance et son typage strict via Protocol Buffers, entre **fleet-service/notification-service** et **auth-service**.

Messagerie asynchrone La diffusion des événements métier (`request.created`, `request.rejected_intern`, `request.resolved`) repose sur **Redis**, utilisé comme file d'attente/bus d'événements (*pub/sub*) : **fleet-service** publie les événements à chaque transition significative du cycle de vie d'une demande, tandis que **notification-service** les consomme en continu de manière découplée et asynchrone.

Notifications push L'envoi effectif des notifications aux utilisateurs (auteur, superviseur et responsable de direction) est délégué à **Firestore Cloud Messaging (FCM)**. Le *Notification Worker* d'**notification-service** résout au préalable les jetons d'appareil (`deviceToken`) via gRPC auprès d'**auth-service**, puis déclenche l'envoi push correspondant à chaque événement consommé.

Persistance des données Les données sont hébergées sur une instance unique **PostgreSQL**, partitionnée en deux schémas cloisonnés : **authentication** (propriété exclusive d'**auth-service**) et **fleet** (propriété exclusive de **fleet-service**). L'accès aux données est assuré via **Spring Data JPA**, sans jointure directe inter-schéma, l'isolation étant garantie au niveau applicatif par les appels gRPC dédiés.

Conteneurisation L'ensemble des services est packagé et orchestré via **Docker** et **Docker Compose**, chaque composant (services applicatifs, **Redis**, **PostgreSQL**, **config-service**, **discovery-service**) s'exécutant dans un conteneur dédié au sein d'un même réseau Docker (*bridge*).

2.2.6 Table de routage (API Gateway)

Le tableau suivant détaille l'ensemble des règles de routage appliquées par **Spring Cloud Gateway**, incluant le préfixe d'URL, le service cible, les rôles autorisés et les contrôles appliqués avant transmission de la requête.

Préfixe	Service cible	Rôles autorisés	Contrôle appliqué	Description
/api/auth/**	auth-service	Public (login) / Authentifié (refresh)	Aucun (login) ; validation JWT (refresh)	Connexion, rafraîchissement de session, gestion des jetons d'appareil
/api/fleet/**	fleet-service	USER, DIVISION_SUPERVISOR, DIVISION_MANAGER, ADMIN	Validation signature JWT (local, via JWKS)	Soumission, consultation, ajustement et validation des demandes ; gestion des lignes
/operator/**	fleet-service (Operator API)	OPERATOR	Validation JWT + vérification claim <code>role=OPERATOR</code>	Accès cloisonné dédié à l'opérateur télécom, filtré par <code>operatorId</code>
/api/admin/**	auth-service	ADMIN	Validation JWT + vérification claim <code>role=ADMIN</code>	Gestion des rôles, permissions, directions et accès opérateur
/api/notifications/**	notification-service	Non exposé (interne)	—	Service asynchrone, aucune exposition directe via la passerelle

Table 1: Table de routage détaillée de l'API Gateway

Ordre d'évaluation des routes Les règles sont évaluées par ordre de spécificité décroissante : `/operator/**` est évalué avant `/api/**` afin d'éviter tout chevauchement avec les routes génériques de **fleet-service**. Une requête ne correspondant à aucun préfixe déclaré est rejetée avec un statut 404 **Not Found**.

Filtres appliqués par route

- **JwtAuthenticationFilter** : extrait et valide la signature du jeton JWT à partir du jeu de clés publiques (JWKS) mis en cache localement, sans appel systématique à **auth-service** ;
- **RoleAuthorizationFilter** : vérifie que le rôle porté par le claim JWT correspond aux rôles autorisés pour la route ciblée, avec rejet 403 **Forbidden** en cas de non-conformité ;
- **OperatorScopeFilter** : filtre spécifique à `/operator/**`, garantissant que l'`operatorId` du jeton correspond strictement aux demandes consultées ou modifiées ;
- **RewritePathFilter** : réécrit le chemin de la requête avant transmission au service cible (suppression du préfixe de routage) ;
- **CircuitBreakerFilter** : applique un mécanisme de repli en cas d'indisponibilité du service cible, avec réponse 503 **Service Unavailable**.

2.2.7 Table des endpoints par service

Le tableau suivant recense l'ensemble des endpoints exposés par chaque service, leur méthode HTTP, les rôles autorisés et une brève description fonctionnelle.

Méthode	Endpoint	Rôles	Description
POST	<code>/api/auth/login</code>	Public	Connexion (CIN + mot de passe), retourne AuthResponse (accessToken, refreshToken, UserSummaryDto)
POST	<code>/api/auth/refresh</code>	Authentifié	Rafraîchissement de session à partir du refreshToken
POST	<code>/api/auth/register</code>	ADMIN	Création d'un utilisateur (RegisterUserRequest)
PUT	<code>/api/auth/device-token</code>	Authentifié	Mise à jour du jeton d'appareil (UpdateDeviceTokenRequest) pour les notifications push

Table 2: Endpoints de l'Auth API

auth-service — Auth API (`/api/auth/**`)

Méthode	Endpoint	Rôles	Description
GET/POST	<code>/api/admin/departments</code>	ADMIN	Consultation et création des directions
GET/POST	<code>/api/admin/users</code>	ADMIN	Gestion des utilisateurs, rôles et rattachements
POST	<code>/api/admin/operators</code>	ADMIN	Création/révocation des accès applicatifs opérateur

Table 3: Endpoints d'administration

auth-service — Administration (`/api/admin/**`)

Méthode	Endpoint	Rôles	Description
POST	/api/requests	USER, DIVISION_SUPERVISOR	Création d'une demande (CreateRequestDto) — état pending_req
PUT	/api/requests/{id}	Auteur	Édition d'une demande en brouillon avant soumission
POST	/api/requests/{id}/submit	Auteur	Soumission de la demande pour validation — état pending_approval
DELETE	/api/requests/{id}	Auteur	Suppression d'une demande en brouillon
GET	/api/requests	USER, DIVISION_SUPERVISOR, DIVISION_MANAGER	Consultation des demandes selon le périmètre (propres demandes / direction)
GET	/api/requests/{id}	USER, DIVISION_SUPERVISOR, DIVISION_MANAGER	Détail d'une demande (RequestDetailDto)
PATCH	/api/requests/{id}/review	DIVISION_MANAGER	Ajustement du contenu (ReviewRequestDto) avant validation
POST	/api/requests/{id}/validate	DIVISION_MANAGER	Validation ou rejet interne (ValidateRequestDto)
GET	/api/lines	USER, DIVISION_SUPERVISOR, DIVISION_MANAGER	Consultation de l'inventaire des lignes (LineDto)
POST	/api/lines/{id}/confirm	USER	Confirmation/enregistrement du numéro reçu par l'utilisateur

Table 4: Endpoints de la Fleet API

fleet-service — Fleet API (/api/)**

Méthode	Endpoint	Rôles	Description
GET	/operator/requests	OPERATOR	Liste des demandes transmises à l'opérateur, filtrées par operatorId (OperatorRequestSummaryDto)
GET	/operator/requests/{id}	OPERATOR	Détail restreint d'une demande transmise
POST	/operator/requests/{id}/respond	OPERATOR	Réponse de traitement (OperatorRespondDto : statut + message), déclenche mise à jour automatique de l'inventaire si COMPLETED

Table 5: Endpoints de l'Operator API

fleet-service — Operator API (/operator/)**

notification-service Le *Notification Worker* n'expose aucun endpoint REST : il consomme en continu les événements publiés sur **Redis** par **fleet-service** et déclenche l'envoi de notifications push (via **FCM**) aux destinataires concernés, après résolution des jetons d'appareil auprès d'**auth-service**. Le tableau suivant détaille, pour chaque événement consommé, les destinataires notifiés et la nature de la notification envoyée.

Événement Redis	Transition	Destinataires	Contenu de la notification
request.created	pending_req → pending_approval	Superviseur de direction (info) ; Responsable de direction (à traiter)	Nouvelle demande soumise par un utilisateur de la direction, en attente de décision
request.rejected_internal	pending_approval → rejected_internal	Auteur de la demande	Rejet de la demande par le responsable de direction, avec motif de rejet joint
request.resolved (statut IN_PROGRESS)	pending_operator (échange sans changement d'état)	Responsable de direction	Message de l'opérateur télécom demandant un complément d'information, sans changement de statut
request.resolved (statut COMPLETED)	pending_operator → completed_operator	Auteur, Superviseur, Responsable de direction	Demande finalisée par l'opérateur ; mise à jour automatique de l'inventaire (ligne créée, modifiée ou désactivée selon le type)
request.resolved (statut REJECTED)	pending_operator → rejected_operator	Auteur, Superviseur, Responsable de direction	Refus de la demande par l'opérateur télécom, avec message explicatif optionnel

Table 6: Notifications déclenchées selon l'état de la demande

Type	Source	Rôles	Description
Consommateur Redis	request.created, request.rejected_internal, request.resolved	—	Aucun endpoint REST exposé ; traitement asynchrone déclenchant l'envoi de notifications push via FCM

Table 7: Interface asynchrone de notification-service (sans exposition REST)

Interface asynchrone (récapitulatif)